Python-tips

In dit onderdeel staan veelgebruikte voorbeelden voor het verwerken, analyseren en visualiseren van meetgegevens met Python. De voorbeelden zijn bedoeld als naslag tijdens de practica.
Wil je direct beginnen? Bij Code snippets staan volledige, zelfstandig uitvoerbare voorbeelden. In dit onderdeel bouwen de korte codefragmenten binnen een onderwerp op elkaar voort.
De belangrijkste packages die we gebruiken zijn:

Package	Veelgebruikt voor	Documentatie
NumPy	Arrays, rekenen en statistiek	NumPy [1]
Matplotlib	Figuren en grafieken	Matplotlib [2]
SciPy	Fits en wetenschappelijke berekeningen	SciPy [3]
Pandas	Tabellen, CSV- en databestanden	Pandas [4]

Packages

Packages worden meestal bovenaan een Python-script of notebook geïmporteerd.

Python
```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from scipy.optimize import curve_fit
```

Voor NumPy, Matplotlib en Pandas worden vrijwel altijd de vaste afkortingen `np`, `plt` en `pd` gebruikt. Hierdoor kunnen functies kort worden aangeroepen:

Python
```python
x = np.array([1, 2, 3, 4, 5])

gemiddelde = np.mean(x)

print(gemiddelde)
```

Je hoeft niet altijd een volledig package te importeren. Met

Python
```python
from scipy.optimize import curve_fit
```

wordt bijvoorbeeld alleen de functie `curve_fit` uit de module `scipy.optimize` geïmporteerd. Deze kan daarna direct worden aangeroepen:

Python
```python
popt, pcov = curve_fit(model, xdata, ydata)
```

Versie controleren

Soms is het handig om te controleren welke versie van een package wordt gebruikt:

Python
```python
print(np.__version__)
print(pd.__version__)
```

Voor een overzicht van de geïnstalleerde packages in de actieve conda-environment kun je in de terminal gebruiken:

Terminal / uitvoer

```
conda list
```

ModuleNotFoundError

Krijg je een foutmelding zoals `ModuleNotFoundError`? Controleer dan eerst of in VS Code de juiste Python-environment of Jupyter-kernel is geselecteerd en of het betreffende package daarin is geïnstalleerd.

Datatypes, lijsten & arrays

Python kent verschillende datatypes. Voor het werken met meetgegevens zijn vooral **lijsten** (`list`) en **NumPy-arrays** (`ndarray`) belangrijk.
Een gewone Python-lijst maak je bijvoorbeeld zo:

Python

```python
tijd = [0, 1, 2, 3, 4]
spanning = [0.2, 1.1, 2.0, 3.2, 4.1]
```

Voor numerieke berekeningen gebruiken we meestal een NumPy-array:

Python

```python
import numpy as np

tijd = np.array([0, 1, 2, 3, 4])
spanning = np.array([0.2, 1.1, 2.0, 3.2, 4.1])
```

Het grote voordeel van arrays is dat berekeningen direct op alle waarden kunnen worden uitgevoerd:

Python

```python
spanning = np.array([1.0, 2.0, 3.0])

spanning_kwadraat = spanning**2

print(spanning_kwadraat)
```

Dit geeft:

Terminal / uitvoer

```
[1. 4. 9.]
```

Bij een Python-lijst werkt een vermenigvuldiging anders:

Python

```python
x = [1, 2, 3]

print(x * 2)
```

Dit geeft:

Terminal / uitvoer

```
[1, 2, 3, 1, 2, 3]
```

Terwijl bij een NumPy-array:

Python

```python
x = np.array([1, 2, 3])

print(x * 2)
```

het resultaat is:

Terminal / uitvoer

```
[2 4 6]
```

Voor het rekenen met meetdata gebruiken we daarom meestal **NumPy-arrays**.

Veelgebruikte datatypes

Type	Voorbeeld	Gebruik
Integer `int`	`5`	Gehele getallen
Float `float`	`5.2`	Getallen met decimalen
Boolean `bool`	`True`, `False`	Logische voorwaarden
String `str`	`"spanning"`	Tekst
List `list`	`[1, 2, 3]`	Verzameling Python-objecten
NumPy-array `ndarray`	`np.array([1, 2, 3])`	Numerieke data en berekeningen

Het datatype van een variabele kun je controleren met:

Python

```python
print(type(spanning))
```

Bij NumPy-arrays kun je daarnaast het datatype van de **elementen** bekijken:

Python

```python
print(spanning.dtype)
```

Efficiënte numerieke berekeningen

Gebruik voor numerieke meetgegevens bij voorkeur NumPy-arrays. Hiermee kun je berekeningen direct op volledige datasets uitvoeren zonder voor iedere bewerking een `for`-loop te schrijven.

Functies

Een **functie** is een stukje code dat een bepaalde bewerking uitvoert. Je geeft een functie één of meerdere waarden mee als **argumenten** en de functie geeft meestal een resultaat terug.
Een eenvoudig voorbeeld is:

Python

```python
x = np.sqrt(9)

print(x)
```

Hier is `np.sqrt()` de functie, `9` het argument en `3` de waarde die de functie teruggeeft.
Een functie kan ook meerdere argumenten hebben:

Python

```python
x = np.round(3.14159, 2)

print(x)
```

Hier krijgt `np.round()` twee argumenten mee: het getal `3.14159` en het gewenste aantal decimalen `2`. NumPy bevat daarnaast veel wiskundige functies die op dezelfde manier direct op arrays kunnen worden toegepast:

Berekening	Python	Documentatie
Wortel	`np.sqrt(x)`	numpy.sqrt [5]
Macht	`np.power(x, n)`	numpy.power [6]
Exponentiële functie e^x	`np.exp(x)`	numpy.exp [7]
Natuurlijke logaritme $\ln(x)$	`np.log(x)`	numpy.log [8]
Logaritme $\log_{10}(x)$	`np.log10(x)`	numpy.log10 [9]
Sinus	`np.sin(x)`	numpy.sin [10]
Cosinus	`np.cos(x)`	numpy.cos [11]
Tangens	`np.tan(x)`	numpy.tan [12]
Arcsinus	`np.arcsin(x)`	numpy.arcsin [13]
Arccosinus	`np.arccos(x)`	numpy.arccos [14]
Arctangens	`np.arctan(x)`	numpy.arctan [15]
Absolute waarde	`np.abs(x)`	numpy.absolute [16]
Afronden	`np.round(x, n)`	numpy.round [17]
Graden → radialen	`np.deg2rad(x)`	numpy.deg2rad [18]
Radialen → graden	`np.rad2deg(x)`	numpy.rad2deg [19]

Radialen & graden

De goniometrische functies van NumPy gebruiken hoeken in **radialen**, niet in graden.
Dit gaat dus **niet goed** wanneer `hoek` in graden staat:

Python
```
hoek = 30
y = np.sin(hoek)        # 30 wordt geïnterpreteerd als 30 radialen
```

Zet de hoek eerst om naar radialen:

Python
```
hoek = 30
hoek_rad = np.deg2rad(hoek)

y = np.sin(hoek_rad)    # sin(30°) = 0.5
```

Dit kan ook direct:

Python
```
y = np.sin(np.deg2rad(30))
```

Een volledig overzicht is te vinden in de NumPy Mathematical functions-documentatie [20].

Indexeren, slicen & filteren

Een enkele waarde uit een array kan worden geselecteerd met een **index**:

Python
```
x = np.array([10, 20, 30, 40])

print(x[0])
print(x[2])
```

Python begint met tellen bij **0**. `x[0]` is dus de eerste waarde en `x[2]` de derde waarde.
Met negatieve indices kun je vanaf het einde van een array tellen:

Python
```
print(x[-1])   # laatste waarde
print(x[-2])   # voorlaatste waarde
```

Een deel van een array selecteren

Met een **slice** kun je meerdere opeenvolgende waarden selecteren:

Python
```
x = np.array([10, 20, 30, 40, 50])

print(x[1:4])
```

Dit geeft:

Terminal / uitvoer
```
[20 30 40]
```

Bij `x[1:4]` wordt index `1` **wel** meegenomen, maar index `4` **niet**.
De algemene notatie is:

Terminal / uitvoer
```
x[start:stop:stap]
```

Bijvoorbeeld:

Python
```
print(x[:3])     # eerste drie waarden
print(x[2:])     # vanaf index 2
print(x[::2])    # iedere tweede waarde
print(x[::-1])   # omgekeerde volgorde
```

Selecteren met een voorwaarde

NumPy kan ook waarden selecteren die aan een bepaalde voorwaarde voldoen:

Python
```
x = np.array([1, 2, 3, 4, 5])

selectie = x[x > 3]

print(selectie)
```

Dit geeft:

Terminal / uitvoer
```
[4 5]
```

Ook meerdere voorwaarden kunnen worden gecombineerd:

Python
```
selectie = x[(x >= 2) & (x <= 4)]

print(selectie)
```

Dit geeft:

Terminal / uitvoer
```
[2 3 4]
```

Gebruik bij NumPy-arrays `&` voor **en** en `|` voor **of**. Zet de afzonderlijke voorwaarden tussen haakjes.

Veelgebruikte selecties en vergelijkingen

Bewerking	Python	Betekenis
Eerste waarde	`x[0]`	Element met index 0
Derde waarde	`x[2]`	Element met index 2
Laatste waarde	`x[-1]`	Laatste element
Eerste drie waarden	`x[:3]`	Index 0 t/m 2
Vanaf derde waarde	`x[2:]`	Vanaf index 2
Deel van array	`x[1:4]`	Index 1 t/m 3
Iedere tweede waarde	`x[::2]`	Stapgrootte 2
Array omkeren	`x[::-1]`	Omgekeerde volgorde
Groter dan	`x[x > 3]`	Waarden groter dan 3
Kleiner dan	`x[x < 3]`	Waarden kleiner dan 3
Groter of gelijk	`x[x >= 3]`	Waarden groter dan of gelijk aan 3
Gelijk aan	`x[x == 3]`	Waarden gelijk aan 3
Niet gelijk aan	`x[x != 3]`	Waarden ongelijk aan 3
Tussen twee waarden	`x[(x >= 2) & (x <= 4)]`	Waarden tussen 2 en 4
Aan één van twee voorwaarden	`x[(x < 2) | (x > 4)]`	Logische OF

Vaste selectie

Dezelfde selectie kan ook op een tweede array worden toegepast. Dit is erg handig bij het verwerken van meetdata.

Python

```
tijd = np.array([0, 1, 2, 3, 4])
spanning = np.array([0.1, 1.2, 2.1, 3.2, 4.1])

selectie = tijd >= 2

tijd_selectie = tijd[selectie]
spanning_selectie = spanning[selectie]
```

Zo blijven de bij elkaar horende waarden van `tijd` en `spanning` gekoppeld wanneer je een deel van de meetdata selecteert.
Meer informatie is te vinden in de NumPy-documentatie over indexing [21].

Beschrijvende statistiek

NumPy bevat functies voor veelgebruikte statistische berekeningen. Bijvoorbeeld voor een serie herhaalde metingen:

Python

```python
import numpy as np

metingen = np.array([4.9, 5.1, 5.0, 5.2, 4.8])

gemiddelde = np.mean(metingen)
mediaan = np.median(metingen)
standaardafwijking = np.std(metingen, ddof=1)

N = len(metingen)
standaardfout = standaardafwijking / np.sqrt(N)

print("Gemiddelde:", gemiddelde)
print("Mediaan:", mediaan)
print("Standaardafwijking:", standaardafwijking)
print("Standaardfout:", standaardfout)
```

Bij experimentele meetreeksen gebruiken we voor de standaardafwijking meestal `ddof=1`, met minimaal twee metingen. Voor onafhankelijke herhaalde metingen onder dezelfde omstandigheden geeft de standaardfout de statistische onzekerheid in het gemiddelde. Systematische onzekerheden, zoals een kalibratiefout, zijn hierin niet opgenomen. De standaardfout wordt berekend met

$$s_{\bar{x}} = \frac{s}{\sqrt{N}}.$$

Handige statistische functies

Berekening	Python	Documentatie
Gemiddelde	`np.mean(x)`	numpy.mean [22]
Mediaan	`np.median(x)`	numpy.median [23]
Standaardafwijking	`np.std(x, ddof=1)`	numpy.std [24]
Variantie	`np.var(x, ddof=1)`	numpy.var [25]
Standaardfout	`np.std(x, ddof=1) / np.sqrt(len(x))`	numpy.std [24]
Minimum	`np.min(x)`	numpy.min [26]
Maximum	`np.max(x)`	numpy.max [27]
Bereik	`np.ptp(x)`	numpy.ptp [28]
Som	`np.sum(x)`	numpy.sum [29]
Aantal waarden	`len(x)`	Python len [30]
Percentiel	`np.percentile(x, p)`	numpy.percentile [31]

Een volledig overzicht van de beschikbare statistische functies is te vinden in de NumPy Statistics-documentatie [32].

Figuren maken

Voor het maken van figuren gebruiken we **Matplotlib**. Een figuur bestaat uit een `figure` en één of meerdere assen (`ax`). De meeste instellingen van de grafiek worden vervolgens via `ax` aangepast.

Eenvoudige lijnplot

Een eenvoudige lijnplot maak je met `ax.plot()`:

Python

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.array([0, 1, 2, 3, 4])
y = np.array([0, 1, 4, 9, 16])

fig, ax = plt.subplots(figsize=(6, 4))

ax.plot(x, y)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Afstand (m)")

fig.tight_layout()
plt.show()
```

Gebruik bij de assen altijd de **grootheid en eenheid**. Gebruik dus bijvoorbeeld `"Tijd (s)"` en `"Spanning (V)"` in plaats van alleen `"x"` en `"y"`.

Meetdata met foutbalken

Voor experimentele meetdata gebruiken we meestal `ax.errorbar()`. Hiermee kunnen de meetonzekerheden direct als foutbalken worden weergegeven.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

xdata = np.array([1, 2, 3, 4, 5])
ydata = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
yerr = np.array([0.1] * 5)

fig, ax = plt.subplots(figsize=(6, 4))

ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    capsize=3
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")

fig.tight_layout()
plt.show()
```

Als ook de x-waarden een onzekerheid hebben, kan `xerr` worden toegevoegd:

Python

```python
xerr = np.array([0.05] * 5)

ax.errorbar(
    xdata,
    ydata,
    xerr=xerr,
    yerr=yerr,
    fmt="o",
    capsize=3
)
```

Lijn versus scatter!

Verbind losse meetpunten niet automatisch met lijnen. Een lijn tussen meetpunten suggereert dat je ook iets weet over de waarden tussen de meetpunten. Gebruik voor experimentele meetdata daarom meestal losse punten met foutbalken. Een theoretisch model of fit kan wel als lijn worden weergegeven.

Veelgebruikte soorten plots

Matplotlib bevat verschillende functies voor het visualiseren van data.

Type	Python	Gebruik	Documentatie
Lijnplot	`ax.plot(x, y)`	Continue functie of model	plot [33]
Meetpunten	`ax.plot(x, y, "o")`	Losse datapunten	plot [33]
Foutbalken	`ax.errorbar(x, y, yerr=dy, fmt="o")`	Meetdata met onzekerheden	errorbar [34]
Scatterplot	`ax.scatter(x, y)`	Relatie tussen twee variabelen	scatter [35]
Histogram	`ax.hist(x)`	Verdeling van meetwaarden	hist [36]
Staafdiagram	`ax.bar(x, y)`	Vergelijken van categorieën	bar [37]
Horizontale lijn	`ax.axhline(y)`	Referentiewaarde aangeven	axhline [38]
Verticale lijn	`ax.axvline(x)`	Referentiewaarde aangeven	axvline [39]

Een volledig overzicht van de mogelijkheden is te vinden in de Matplotlib-documentatie [40].

Veelgebruikte instellingen

De eigenschappen van een figuur kunnen via `ax` worden aangepast:

Instelling	Python
Label x-as	`ax.set_xlabel("Tijd (s)")`
Label y-as	`ax.set_ylabel("Spanning (V)")`
Titel	`ax.set_title("Meting")`
Legenda	`ax.legend()`
Raster	`ax.grid()`
x-bereik	`ax.set_xlim(xmin, xmax)`
y-bereik	`ax.set_ylim(ymin, ymax)`
Logaritmische x-as	`ax.set_xscale("log")`
Logaritmische y-as	`ax.set_yscale("log")`
Minor ticks	`ax.minorticks_on()`
Figuur netjes uitlijnen	`fig.tight_layout()`

Bij meerdere datasets of een combinatie van meetdata en een model kan met `label` een legenda worden gemaakt:

Python

```
ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    capsize=3,
    label="Meetdata"
)

ax.plot(x, y, label="Model")

ax.legend()
```

Een publicatiewaardige figuur

Een figuur voor een verslag, poster of presentatie moet **zelfstandig leesbaar** zijn. De lezer moet kunnen zien welke grootheden zijn weergegeven, welke onzekerheden bij de metingen horen en welke onderdelen meetdata of modelresultaten zijn.
Een goed uitgangspunt is:

Python

```python
import numpy as np
import matplotlib.pyplot as plt

xdata = np.array([1, 2, 3, 4, 5])
ydata = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
yerr = np.array([0.1] * 5)

fig, ax = plt.subplots(figsize=(6, 4))

ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    markersize=5,
    capsize=3,
    linewidth=1,
    label="Meetdata"
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")

ax.tick_params(
    direction="in",
    top=True,
    right=True
)

ax.minorticks_on()

ax.legend(frameon=False)

fig.tight_layout()
plt.show()
```

Aanbevolen instellingen

Onderdeel	Instelling	Opmerking
Figuurgrootte	`figsize=(6, 4)`	Goed uitgangspunt voor een losse figuur
Meetpunten	`fmt="o"`	Duidelijke losse meetpunten
Puntgrootte	`markersize=5`	Niet te groot en niet te klein
Foutbalken	`capsize=3`	Maakt foutbalken beter leesbaar
Lijndikte	`linewidth=1`	Rustige, duidelijke lijnen
Assenlabels	`set_xlabel()`, `set_ylabel()`	Altijd grootheid én eenheid
Tickrichting	`direction="in"`	Veelgebruikt in wetenschappelijke figuren
Ticks boven/rechts	`top=True`, `right=True`	Geeft een rustig afgesloten assenstelsel
Minor ticks	`minorticks_on()`	Handig voor nauwkeurig aflezen
Legenda	`legend(frameon=False)`	Alleen nodig bij meerdere datasets of modellen
Uitlijning	`tight_layout()`	Voorkomt afgesneden labels

Lettergrootte instellen

Voor een verslag is het handig om de lettergroottes expliciet in te stellen:

Python

```python
plt.rcParams.update({
    "font.size": 11,
    "axes.labelsize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10
})
```

Zet dit bij voorkeur bovenaan het script, vóór het maken van de figuur.

Figuur opslaan

Voor een verslag heeft een vectorformaat zoals PDF meestal de voorkeur:

Python

```python
fig.savefig(
    "figuur.pdf",
    bbox_inches="tight"
)
```

Voor PNG:

Python

```python
fig.savefig(
    "figuur.png",
    dpi=300,
    bbox_inches="tight"
)
```

Gebruik voor een publicatiewaardige rasterafbeelding bij voorkeur minimaal `dpi=300`.

Goede figuren

Een goede wetenschappelijke figuur bevat alleen informatie die nodig is om de data te begrijpen. Gebruik dus geen decoratieve achtergronden, overmatige kleuren, dikke rasterlijnen of 3D-effecten.

Een goede figuur bevat in ieder geval:

1. duidelijk leesbare assen;
2. grootheden en eenheden bij de assen;
3. foutbalken wanneer meetonzekerheden bekend zijn;
4. een legenda wanneer meerdere datasets of modellen worden weergegeven;
5. voldoende grote tekst en symbolen;
6. geen onnodige visuele elementen.

Titels, legenda's en onderschriften

Gebruik een onderschrift om de meting, het model en de betekenis van de foutbalken toe te lichten. Een titel is vaak overbodig wanneer het onderschrift dezelfde informatie geeft. Een legenda is wel nuttig om meerdere datasets of modellen te onderscheiden. Houd assenlabels en eenheden in de figuur zelf.

Een model fitten

Wanneer uit de theorie een wiskundig model volgt voor het verband tussen twee grootheden, kunnen we de onbekende parameters van dat model bepalen door het model aan de meetdata te **fitten**.

Bij een fit zoekt Python naar de waarden van de modelparameters waarvoor het model zo goed mogelijk overeenkomt met de meetgegevens, rekening houdend met de opgegeven meetonzekerheden.

Stel dat we op basis van de theorie een lineair verband verwachten:

$$y = ax + b.$$

Hierbij zijn:

- x en y de gemeten grootheden;
- a de richtingscoëfficiënt;
- b het snijpunt met de y-as.

De parameters a en b zijn dus de onbekende grootheden die we met de fit willen bepalen.

Stap 1 — Het model definiëren

We schrijven het theoretische model eerst als Python-functie:

Python

```python
def linear(x, a, b):
    return a * x + b
```

De eerste invoer `x` is de onafhankelijke variabele. De overige invoeren, hier `a` en `b`, zijn de parameters die door de fit moeten worden bepaald.

Stap 2 — Meetdata invoeren

We gebruiken als voorbeeld de volgende meetgegevens:

```python
import numpy as np
from scipy.optimize import curve_fit

xdata = np.array([1, 2, 3, 4, 5])
ydata = np.array([1.1, 1.9, 3.2, 3.9, 5.1])

yerr = np.array([0.1] * 5)
```

Hier bevat `yerr` de positieve standaardonzekerheid (één sigma) van iedere gemeten y-waarde. We nemen onafhankelijke meetfouten aan. `curve_fit` neemt onzekerheden in de x-waarden niet mee: ook het tekenen van `xerr` verandert dat niet.

Stap 3 — De fit uitvoeren

De fit voeren we uit met `curve_fit()` uit SciPy:

```python
p0 = [1, 0]

popt, pcov = curve_fit(
    linear,
    xdata,
    ydata,
    sigma=yerr,
    p0=p0,
    absolute_sigma=True
)
```

De belangrijkste argumenten zijn:

Argument	Betekenis
`linear`	het model dat wordt gefit
`xdata`	de gemeten waarden van x
`ydata`	de gemeten waarden van y
`sigma=yerr`	de onzekerheden van de y-metingen
`p0=[1, 0]`	eerste schatting van de parameters a en b
`absolute_sigma=True`	zorgt ervoor dat `yerr` als absolute meetonzekerheid wordt gebruikt

De functie geeft twee resultaten terug:

```python
popt
pcov
```

`popt` bevat de gevonden optimale waarden van de fitparameters. In dit voorbeeld geldt:

```python
a = popt[0]
b = popt[1]
```

`pcov` is de covariantiematrix van de fitparameters. Hieruit kunnen we de onzekerheden van a en b bepalen.
Bij een niet-lineair model is deze onzekerheidsschatting gebaseerd op een lokale lineaire benadering. Zonder `absolute_sigma=True` schaalt SciPy de covariantiematrix met de gereduceerde chi-kwadraat. Zie de documentatie van curve_fit [41].

Volgorde van de parameters

De volgorde in `popt` is dezelfde als de volgorde van de parameters in de modeldefinitie.
Bij

Python
```
def linear(x, a, b):
    return a * x + b
```

geldt dus:

Python
```
popt[0]   # a
popt[1]   # b
```

Onzekerheid van de fitparameters

Een fit levert niet alleen de beste waarden van de parameters op, maar ook informatie over hun onzekerheid.
De standaardonzekerheden van de fitparameters volgen uit de diagonaalelementen van `pcov`:

Python
```
perr = np.sqrt(np.diag(pcov))

a = popt[0]
b = popt[1]

a_err = perr[0]
b_err = perr[1]

print("a =", a, "+-", a_err)
print("b =", b, "+-", b_err)
```

Voor deze meetdata vinden we ongeveer:

$$a = 1.000 \pm 0.032$$

en

$$b = 0.040 \pm 0.105.$$

Het resultaat van de fit rapporteer je dus inclusief de onzekerheid van de gevonden parameters.

Onzekerheid in x

`curve_fit()` verwerkt met `sigma=` alleen onzekerheden in de y-waarden.
Als de onzekerheid in x niet verwaarloosbaar is, is deze eenvoudige aanpak niet voldoende.

Fit en meetdata samen plotten

Om de fit te beoordelen is het belangrijk om zowel de meetdata als het gefitte model te bekijken.
De meetdata tekenen we als losse punten met foutbalken. Voor het model maken we een fijn verdeelde reeks x-waarden, zodat de fit als een vloeiende lijn wordt weergegeven.

Python

```python
import matplotlib.pyplot as plt

xfit = np.linspace(
    np.min(xdata),
    np.max(xdata),
    200
)

yfit = linear(xfit, *popt)

fig, ax = plt.subplots(figsize=(6, 4))

ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    capsize=3,
    label="Meetdata"
)

ax.plot(
    xfit,
    yfit,
    label="Lineaire fit"
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")

ax.tick_params(
    direction="in",
    top=True,
    right=True
)

ax.minorticks_on()
ax.legend(frameon=False)

fig.tight_layout()
plt.show()
```

De uitdrukking

Python

```python
linear(xfit, *popt)
```

is een korte schrijfwijze voor:

Python

```python
linear(xfit, popt[0], popt[1])
```

Het sterretje `*` pakt de waarden uit `popt` uit en geeft ze afzonderlijk door aan de functie.

Meetpunten en fitlijn

De 200 waarden in `xfit` zijn **geen extra meetpunten**.
Ze worden alleen gebruikt om het wiskundige model als een vloeiende lijn weer te geven.

De kwaliteit van een fit beoordelen

Het feit dat Python een fit kan uitvoeren betekent nog niet automatisch dat het gekozen model de meetdata goed beschrijft.
Een eerste stap is daarom altijd:

1. bekijk de meetpunten en foutbalken;
2. bekijk de fitlijn;
3. controleer of er systematische afwijkingen zichtbaar zijn;
4. bereken eventueel de chi-kwadraat en bekijk de residuen.

Chi-kwadraat

De chi-kwadraat vergelijkt de afwijking tussen iedere meetwaarde en het model met de onzekerheid van die meting:

$$\chi^2 = \sum_i \left(\frac{y_i - y_{\text{model},i}}{\sigma_i} \right)^2.$$

Voor ieder meetpunt wordt dus gekeken naar:

$$\frac{y_i - y_{\text{model},i}}{\sigma_i} = \frac{\text{afwijking tussen meting en model}}{\text{onzekerheid van de meting}}.$$

Een afwijking van bijvoorbeeld 0.2 is veel belangrijker wanneer de meetonzekerheid 0.01 is dan wanneer de meetonzekerheid 0.5 is.
In Python:

Python

```
y_model = linear(xdata, *popt)

chi2 = np.sum(
    ((ydata - y_model) / yerr)**2
)

print("chi^2 =", chi2)
```

Gereduceerde chi-kwadraat

De gewone χ^2 hangt onder andere af van het aantal meetpunten. Daarom gebruiken we vaak de **gereduceerde chi-kwadraat**:

$$\chi^2_{\text{red}} = \frac{\chi^2}{N - p}.$$

Hierbij is:

- N het aantal meetpunten;
- p het aantal vrije fitparameters;
- $N - p$ het aantal vrijheidsgraden.

Voor het lineaire model zijn er twee fitparameters, namelijk a en b. Bij vijf meetpunten geldt dus:

$$N - p = 5 - 2 = 3.$$

In Python:

Python

```
dof = len(xdata) - len(popt)

chi2_red = chi2 / dof

print("Vrijheidsgraden =", dof)
print("chi^2_red =", chi2_red)
```

Voor de voorbeelddata vinden we ongeveer:

Grootheid	Waarde
a	1.000 ± 0.032
b	0.040 ± 0.105
χ^2	7.20
Vrijheidsgraden $N - p$	3
χ^2_{red}	2.40

Bij onafhankelijke, normaal verdeelde meetfouten met correct opgegeven standaardonzekerheden en een geschikt model verwachten we een waarde van χ^2_{red} van **orde 1**. Hiervoor moet $N > p$ gelden. Bij weinig vrijheidsgraden kan de waarde sterk fluctueren; voor een lineair model is de standaardafwijking van deze grootheid $\sqrt{2/(N-p)}$.
Een veel grotere waarde kan bijvoorbeeld betekenen dat:

- het gekozen model de meetdata niet goed beschrijft;
- de meetonzekerheden te klein zijn geschat;
- er systematische effecten aanwezig zijn die niet in het model zijn opgenomen.

Een veel kleinere waarde kan bijvoorbeeld betekenen dat de meetonzekerheden ruim zijn geschat.

Geen automatische goed/fout-test

Gebruik χ^2_{red} niet als automatische grens tussen een goede en slechte fit.
Bekijk altijd ook de meetpunten, foutbalken, fitlijn en residuen.

Residuen bekijken

De **residuen** zijn de verschillen tussen de gemeten waarden en de waarden die het model voorspelt:

$$\epsilon_i = y_i - y_{\text{model},i}.$$

In Python:

Python

```
y_model = linear(xdata, *popt)

residuen = ydata - y_model

print(residuen)
```

Een positief residu betekent dat de meetwaarde **boven** het model ligt. Een negatief residu betekent dat de meetwaarde **onder** het model ligt.
De residuen kunnen ook worden geplot:

Python

```python
fig, ax = plt.subplots(figsize=(6, 3))

ax.errorbar(
    xdata,
    residuen,
    yerr=yerr,
    fmt="o",
    capsize=3
)

ax.axhline(
    0,
    linewidth=1
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Residu (m)")

ax.tick_params(
    direction="in",
    top=True,
    right=True
)

ax.minorticks_on()

fig.tight_layout()
plt.show()
```

Bij een geschikt model verwacht je dat de residuen zonder duidelijk systematisch patroon rond nul verspreid liggen.
Bijvoorbeeld:

- willekeurig boven en onder nul → model kan geschikt zijn;
- een kromming in de residuen → mogelijk is het gekozen model te eenvoudig;
- residuen steeds aan één kant van nul → mogelijk is er een systematische afwijking.

Een fit beoordelen

Kijk bij het beoordelen van een fit dus niet alleen naar de gevonden fitparameters.
Gebruik in ieder geval:
meetdata + foutbalken → fitlijn → residuen → χ^2_{red}.

Data inlezen uit een CSV-bestand

Meetgegevens worden vaak opgeslagen in een **CSV-bestand**. Met Pandas kan zo'n bestand eenvoudig worden ingelezen:

Python

```python
import pandas as pd

data = pd.read_csv("metingen.csv")
```

Een kolom kan vervolgens worden geselecteerd met:

Python
```
tijd = data["tijd"]
spanning = data["spanning"]
```

Instellingen voor `read_csv()`

Met onderstaande instellingen geef je aan **hoe het bestand is opgebouwd**:

Instelling	Betekenis	Voorbeeld
`sep`	Scheidingsteken tussen kolommen	`sep=";"`
`decimal`	Decimaalteken	`decimal=","`
`header`	Welke regel de kolomnamen bevat	`header=0`
`names`	Zelf kolomnamen opgeven	`names=["tijd", "spanning"]`
`skiprows`	Aantal regels bovenaan overslaan	`skiprows=2`
`usecols`	Alleen bepaalde kolommen inlezen	`usecols=["tijd", "spanning"]`
`na_values`	Waarden die als ontbrekende data moeten worden gezien	`na_values=["-", "N/A"]`
`comment`	Teken waarmee commentaarregels beginnen	`comment="#"`

Bekijk voor alle beschikbare instellingen de documentatie van Pandas [42].
Een CSV-bestand uit Nederlandse software gebruikt bijvoorbeeld vaak een **puntkomma als scheidingsteken** en een **komma als decimaalteken**:

Terminal / uitvoer
```
tijd;spanning;onzekerheid
0;0,12;0,02
1;1,05;0,02
2;2,08;0,03
```

Dit lees je in met:

Python
```
data = pd.read_csv(
    "metingen.csv",
    sep=";",
    decimal=","
)
```

Controleer de data

Controleer na het inlezen altijd kort of de kolommen en getallen correct zijn ingelezen:

Python
```
print(data.head())
print(data.dtypes)
```

Worden getallen niet als numerieke waarden herkend? Controleer dan eerst `sep` en `decimal`.

Veelvoorkomende fouten

Een foutmelding in Python bevat meestal informatie over **wat er fout gaat en waar de fout optreedt**. Kijk daarom altijd eerst naar de laatste regel van de foutmelding. Hier staat meestal het type fout, bijvoorbeeld `NameError`, `ValueError` of `FileNotFoundError`.

`NameError`

Bijvoorbeeld:

Terminal / uitvoer

```
NameError: name 'np' is not defined
```

Python kent de naam `np` niet. Waarschijnlijk is NumPy nog niet geïmporteerd:

Python

```python
import numpy as np
```

Een `NameError` kan ook ontstaan door een typefout in een variabelenaam:

Python

```python
spanning = 5.0

print(spaning)  # typefout
```

NameError

Controleer bij een `NameError` of de variabele of functie al is gedefinieerd en of de naam correct is gespeld.

`ModuleNotFoundError`

Bijvoorbeeld:

Terminal / uitvoer

```
ModuleNotFoundError: No module named 'scipy'
```

Python kan het package niet vinden. Controleer eerst of in VS Code de juiste **Python-environment of Jupyter-kernel** is geselecteerd.
Wanneer SciPy nog niet in de gebruikte conda-environment staat, installeer je het vanuit de terminal met:

Terminal / uitvoer

```
conda install scipy
```

Je kunt controleren welke packages in de actieve environment zijn geïnstalleerd met:

Terminal / uitvoer

```
conda list
```

`FileNotFoundError`

Bijvoorbeeld:

Terminal / uitvoer

```
FileNotFoundError: [Errno 2] No such file or directory: 'metingen.csv'
```

Python kan het opgegeven bestand niet vinden.
Controleer:

- of de bestandsnaam correct is;
- of de extensie klopt, bijvoorbeeld `.csv`;
- of het bestand in de juiste map staat;
- of het juiste bestandspad is opgegeven.

Een relatief bestandspad wordt geïnterpreteerd ten opzichte van de **huidige werkmap**, niet automatisch de map van je script. Bekijk deze met `from pathlib import Path` en `print(Path.cwd())`.
Staat het bestand in de huidige werkmap:

Python
```
data = pd.read_csv("metingen.csv")
```

Staat het bestand in een submap `data`:

Python
```
data = pd.read_csv("data/metingen.csv")
```

`KeyError`

Bijvoorbeeld:

Terminal / uitvoer
```
KeyError: 'spanning'
```

Dit komt bij Pandas vaak voor wanneer je een kolom probeert te selecteren die niet bestaat:

Python
```
spanning = data["spanning"]
```

Bekijk de beschikbare kolomnamen met:

Python
```
print(data.columns)
```

Controleer daarbij ook op hoofdletters en spaties. `"Spanning"` en `"spanning"` zijn voor Python verschillende namen.

`TypeError`

Bijvoorbeeld:

Terminal / uitvoer
```
TypeError: can't multiply sequence by non-int of type 'float'
```

Een `TypeError` betekent dat een bewerking wordt uitgevoerd op een datatype waarvoor die bewerking niet is toegestaan.
Controleer het datatype met:

Python
```
print(type(x))
```

Bij meetgegevens kan het bijvoorbeeld nodig zijn om gegevens om te zetten naar een numeriek datatype:

Python
```
x = np.array(x, dtype=float)
```

`ValueError`

Bijvoorbeeld:

Terminal / uitvoer
```
ValueError: could not convert string to float
```

Python verwacht een getal, maar vindt bijvoorbeeld tekst.
Dit komt bij meetbestanden onder andere voor wanneer getallen verkeerd worden ingelezen. Controleer dan bijvoorbeeld het **decimaalteken** en **scheidingsteken**:

Python
```
data = pd.read_csv(
    "metingen.csv",
    sep=";",
    decimal=","
)
```

Controleer eventueel de datatypes:

Python
```
print(data.dtypes)
```

`IndexError`

Bijvoorbeeld:

Terminal / uitvoer
```
IndexError: index 5 is out of bounds
```

Je probeert een element uit een array op te vragen dat niet bestaat.
Bijvoorbeeld:

Python
```
x = np.array([10, 20, 30])

print(x[3])
```

Deze array heeft drie elementen, maar de indices zijn:

Terminal / uitvoer
```
0, 1, 2
```

Python begint bij index `0`.

Arrays hebben verschillende lengtes

Bij het maken van een figuur kan bijvoorbeeld de volgende fout optreden:

Terminal / uitvoer
```
ValueError: x and y must have same first dimension
```

De arrays met x- en y-waarden hebben dan niet hetzelfde aantal elementen.
Controleer dit met:

Python
```
print(len(xdata))
print(len(ydata))
```

Voor iedere x-waarde moet er een bijbehorende y-waarde zijn.
Als je per meetpunt een foutbalk opgeeft, controleer dan ook:

Python
```
print(len(xdata))
print(len(ydata))
print(len(yerr))
```

Voor gelijke foutbalken mag `yerr` ook één getal zijn, bijvoorbeeld `yerr=0.1`. In dat geval is `len(yerr)` niet van toepassing.

`SyntaxError`

Bijvoorbeeld:

Terminal / uitvoer

```
SyntaxError: invalid syntax
```

Python kan de geschreven code niet interpreteren. Veelvoorkomende oorzaken zijn:

Python

```python
print("Hallo"       # haakje ontbreekt

x = [1, 2, 3       # blokhaakje ontbreekt

if x > 2           # dubbele punt ontbreekt
```

Controleer bij een `SyntaxError` vooral de regel die Python aanwijst én de regel direct daarvoor.

Foutmeldingen lezen

Kijk bij een foutmelding eerst naar de **laatste regel**. Daar staat meestal de belangrijkste informatie. Bijvoorbeeld:

Terminal / uitvoer

```
FileNotFoundError: [Errno 2] No such file or directory: 'metingen.csv'
```

Hier vertelt Python zowel **welk type fout** optreedt (`FileNotFoundError`) als **wat het probleem is** (`metingen.csv` kan niet worden gevonden).

Documentatie voor op papier

De nummers bij de links verwijzen naar onderstaande webadressen.

1. https://numpy.org/doc/stable/
2. https://matplotlib.org/stable/
3. https://docs.scipy.org/doc/scipy/
4. https://pandas.pydata.org/docs/
5. https://numpy.org/doc/stable/reference/generated/numpy.sqrt.html
6. https://numpy.org/doc/stable/reference/generated/numpy.power.html
7. https://numpy.org/doc/stable/reference/generated/numpy.exp.html
8. https://numpy.org/doc/stable/reference/generated/numpy.log.html
9. https://numpy.org/doc/stable/reference/generated/numpy.log10.html
10. https://numpy.org/doc/stable/reference/generated/numpy.sin.html
11. https://numpy.org/doc/stable/reference/generated/numpy.cos.html
12. https://numpy.org/doc/stable/reference/generated/numpy.tan.html
13. https://numpy.org/doc/stable/reference/generated/numpy.arcsin.html
14. https://numpy.org/doc/stable/reference/generated/numpy.arccos.html
15. https://numpy.org/doc/stable/reference/generated/numpy.arctan.html
16. https://numpy.org/doc/stable/reference/generated/numpy.absolute.html
17. https://numpy.org/doc/stable/reference/generated/numpy.round.html
18. https://numpy.org/doc/stable/reference/generated/numpy.deg2rad.html
19. https://numpy.org/doc/stable/reference/generated/numpy.rad2deg.html
20. https://numpy.org/doc/stable/reference/routines.math.html
21. https://numpy.org/doc/stable/user/basics.indexing.html
22. https://numpy.org/doc/stable/reference/generated/numpy.mean.html
23. https://numpy.org/doc/stable/reference/generated/numpy.median.html
24. https://numpy.org/doc/stable/reference/generated/numpy.std.html
25. https://numpy.org/doc/stable/reference/generated/numpy.var.html
26. https://numpy.org/doc/stable/reference/generated/numpy.min.html
27. https://numpy.org/doc/stable/reference/generated/numpy.max.html
28. https://numpy.org/doc/stable/reference/generated/numpy.ptp.html

29 https://numpy.org/doc/stable/reference/generated/numpy.sum.html
30 https://docs.python.org/3/library/functions.html#len
31 https://numpy.org/doc/stable/reference/generated/numpy.percentile.html
32 https://numpy.org/doc/stable/reference/routines.statistics.html
33 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.plot.html
34 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.errorbar.html
35 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.scatter.html
36 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.hist.html
37 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.bar.html
38 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.axhline.html
39 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.axvline.html
40 https://matplotlib.org/stable/plot_types/index.html
41 https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
42 https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html